

README - Automatización de Despliegue Serverless en AWS

1. Nombre del Proyecto

Serverless Deployment Automation

Automatización de despliegue para una función Lambda expuesta mediante API Gateway usando **AWS SAM** y CI/CD con **GitHub Actions**.

2. Objetivo

Este proyecto busca automatizar el despliegue de una aplicación serverless en AWS de forma **repetible, confiable y segura**, incorporando controles de calidad, seguridad y trazabilidad.

La aplicación consiste en un endpoint HTTP GET `/hello` que retorna:

```
{"message": "hello world"}
```

3. Repositorio Base

El repositorio incluye:

- `template.yaml` → Template AWS SAM de la aplicación.
 - `hello_world/` → Código fuente de la función Lambda.
 - `requirements.txt` → Dependencias Python necesarias.
 - `.github/workflows/deploy.yml` → Pipeline de CI/CD en GitHub Actions.
 - `README.md` → Documentación y guía de uso.
-

4. Estrategia de Ramas

Se implementa un flujo simple y claro:

Rama	Propósito	Acciones CI/CD
------	-----------	----------------

<code>devel op</code>	Desarrollo de nuevas funciones / cambios	Pipeline corre build y pruebas, no despliega
<code>main</code>	Código estable listo para producción	Pipeline corre build, pruebas y despliegue automático

Flujo recomendado:

1. Hacer cambios en `develop`.
2. Abrir Pull Request hacia `main`.
3. Revisar pruebas y validaciones.
4. Merge a `main` → pipeline despliega automáticamente en AWS.

5. Configuración Requerida

5.1 Secrets en GitHub

Para el pipeline de GitHub Actions se deben configurar los siguientes secrets:

- `AWS_REGION` → Región AWS donde se desplegará la aplicación (ej. `us-east-1`).
- `AWS_ROLE_ARN` → ARN del rol IAM que el pipeline usará para desplegar con permisos mínimos.

5.2 Requisitos locales

Si se desea probar localmente:

- Python ≥ 3.12
- AWS CLI configurado
- AWS SAM CLI instalado
- Dependencias Python:

```
pip install -r requirements.txt
```

6. Pipeline de CI/CD

Archivo: `.github/workflows/deploy.yml`

Etapas del pipeline

1. Checkout del código

```
uses: actions/checkout@v4
```

2. Configuración de entorno

- Python 3.12
- Variables de entorno (`PYTHONPATH`, `AWS_REGION`)

3. Instalación de dependencias

```
pip install -r requirements.txt
pip install aws-sam-cli
```

4. Ejecución de pruebas

```
pytest
```

5. Validación del template SAM

```
sam validate --region $AWS_REGION
```

6. Build de la aplicación

```
sam build
```

7. Despliegue seguro en AWS *(solo rama main)*

```
sam deploy \
--no-confirm-changeset \
--no-fail-on-empty-changeset \
--stack-name sam-hello-world \
--capabilities CAPABILITY_IAM \
--resolve-s3 \
--region $AWS_REGION
```

8. Validación post-despliegue

- Obtiene el endpoint API Gateway desde CloudFormation.
- Ejecuta `curl` para verificar que devuelve `"hello world"`.

7. Ejecución Manual del Despliegue

Para desplegar manualmente:

```
sam build
sam validate
```

```
sam deploy --stack-name sam-hello-world --capabilities CAPABILITY_IAM --resolve-s3  
--region us-east-1
```

Validar endpoint:

```
URL=$(aws cloudformation describe-stacks \  
--stack-name sam-hello-world \  
--query "Stacks[0].Outputs[?OutputKey=='HelloApi'].OutputValue" \  
--output text)
```

```
curl $URL
```

Respuesta esperada:

```
{"message": "hello world"}
```

8. Pruebas y Validaciones

- **Unit tests:** Ejecutados con `pytest`.
- **Validación de template SAM:** `sam validate`
- **Post-deploy check:** `curl` al endpoint `/hello` para confirmar que responde correctamente.
- **Logs de Lambda** (opcional):

```
sam logs -n HelloWorldFunction --stack-name sam-hello-world
```

9. Seguridad

- Pipeline usa **OIDC + rol IAM** → no se almacenan claves estáticas.
 - Rol IAM tiene **permisos mínimos necesarios** para Lambda, CloudFormation, S3 y API Gateway.
 - Variables sensibles gestionadas mediante **GitHub Secrets**.
 - Rama `main` protegida para evitar despliegues accidentales.
-

10. Buenas Prácticas Implementadas

- Separación clara entre **build** → **test** → **deploy**.
- Variables de entorno y región configurables.
- Validación de template antes del despliegue.
- Validación post-deploy del endpoint.

- Estrategia de ramas clara y reproducible.
 - Pipeline seguro y repetible.
-

11. Resumen de Entrega

Entregable	Estado
Repositorio base AWS SAM	✓ Hecho
Pipeline CI/CD implementado	✓ Hecho
Deploy automático desde main	✓ Hecho
Validación endpoint	✓ Hecho
Estrategia de ramas documentada	✓ Hecho
README con instrucciones	✓ Hecho

12. Próximos pasos (Opcional)

- Agregar pruebas unitarias más complejas para Lambda.
- Monitoreo de logs en CloudWatch desde pipeline.
- Alertas de despliegue fallido (GitHub Actions / SNS).